

Hledání podobných hashů

Hash nějakého řetězce je unikátní hodnota, která je výsledkem hashovací funkce. Hashovací funkce jsou funkce, která převádějí vstupní data na výsledný řetězec, který má často **pevnou délku**.

Některé z nejznámějších hashovacích funkcí jsou například:

- MD5
- SHA-1
- SHA-256

Příklad:

```
MD5("Hello World") = "b10a8db164e0754105b7a99be72e3fe5"
SHA-1("Hello World") = "0a4d55a8d778e5022fab701977c5d840bbc486d0"
SHA-256("Hello World") =
"a591a6d40bf420404a011733cfeb7b190d62c65bf0bcda32b57b277d9ad9f146e"
```

Navrhne si vlastní hashovací funkci, která bude vracet hash jako řetězec o délce 16 znaků. Naše hashovací funkce bude jednodušší, než ty, které jsme zmínili výše. Omezíme se také pouze na hodnoty z klasické ASCII tabulky (0-127). Výsledný řetězec budeme chtít zkonstruovat pouze z malých písmen anglické abecedy.

Popis hashovací funkce

Algoritmus hashe je následující:

- inicializuj `result` na řetězec `"0123456789abcdef"`
- pro každé dva sousední znaky `(a, b)` vstupního řetězce se provede
 - `sum` = součet jejich ASCII hodnot = `ascii(a) + ascii(b)`
 - `tmp` = `sum modulo 128` (zbytek po dělení číslem 128 -> hodnota v rozsahu 0-127)
 - `pos` = `sum modulo 16` (zbytek po dělení číslem 16 -> hodnota v rozsahu 0-15)
 - `result[pos] = (ascii(result[pos]) + tmp) mod 128` (zbytek po dělení číslem 128 -> hodnota v rozsahu 0-127)
 - cyklicky posunout `result` o `pos` pozic doprava (např. posunutí "1234" o 2 pozice doprava -> "3412")
- pro každý znak `c` na pozici `pos` v `result` se provede
 - `result[pos] = 97 + (ascii(c) mod 26)` (97 je ASCII hodnota malého písmene 'a' a 26 je počet malých písmen -> hodnota v rozsahu 97-122, což odpovídá malým písmenům anglické abecedy)
- výsledkem je řetězec, který obsahuje 16 malých písmen anglické abecedy

Příklad výpočtu hashe pro vstupní řetězec `"Hello World"` :

- `ascii('H') = 72`
- `ascii('e') = 101`
- `ascii('l') = 108`
- `ascii('l') = 108`
- `ascii('o') = 111`
- `ascii(' ') = 32`

- `ascii('W') = 87`
- `ascii('o') = 111`
- `ascii('r') = 114`
- `ascii('l') = 108`
- `ascii('d') = 100`

Iterace 1, dvojice (H, e):

- `sum = 72 + 101 = 173`
- `tmp = 173 mod 128 = 45`
- `pos = 173 mod 16 = 13`
- `result[13] = (ascii(result[13]) + 45) mod 128 = (ascii('d') + 45) mod 128 = (100 + 45) mod 128 = 145 mod 128 = 17`
- **!! (na indexu 13 se aktuálně nachází netisknutelný znak s ASCII hodnotou 17)**
- `result = posun("0123456789abc ef", 13) = "3456789abc ef012"`

Iterace 2, dvojice (e, l):

- `sum = 101 + 108 = 209`
- `tmp = 209 mod 128 = 81`
- `pos = 209 mod 16 = 1`
- `result[1] = (ascii(result[1]) + 81) mod 128 = (ascii('4') + 81) mod 128 = (52 + 81) mod 128 = 133 mod 128 = 5`
- **!! (na indexu 1 se aktuálně nachází netisknutelný znak s ASCII hodnotou 5)**
- `result = posun("3 56789abc ef012", 1) = "23 56789abc ef01"`

Iterace 3, dvojice (l, l):

- `sum = 108 + 108 = 216`
- `tmp = 216 mod 128 = 88`
- `pos = 216 mod 16 = 8`
- `result[8] = (ascii(result[8]) + 88) mod 128 = (ascii('a') + 88) mod 128 = (97 + 88) mod 128 = 185 mod 128 = 57`
- **!! (na indexu 8 se aktuálně nachází znak s ASCII hodnotou 57, což odpovídá znaku '9')**
- `result = posun("23 567899bc ef01", 8) = "9bc ef0123 56789"`

Další iterace se provádějí obdobně. Poslední iterace vypadá následovně:

Iterace 10, dvojice (l, d):

- `sum = 108 + 100 = 208`
- `tmp = 208 mod 128 = 80`
- `pos = 208 mod 16 = 0`
- `result[0] = (ascii(result[0]) + 80) mod 128 = (ascii('f') + 80) mod 128 = (102 + 80) mod 128 = 182 mod 128 = 54`
- **!! (na indexu 0 se aktuálně nachází znak s ASCII hodnotou 54, což odpovídá znaku '6')**
- `result = posun("6 6.899bc t, 123", 0) = "6 6.899bc t, 123"`

Na tento `result` aplikujeme poslední krok algoritmu, kde se výsledné znaky převedou na malá písmena anglické abecedy a získáme výsledný hash.

Výsledný hash pro vstupní řetězec `Hello World` je `"cqcueffuvrmsoxyz"`.

Hledání (částečných) kolizí

Hashovací funkce jsou jednosměrná, což znamená, že z hashe by **nemělo být možné** zpětně získat původní vstupní řetězec.

Tento problém také souvisí s problematikou **kolizí**. Kolize nastává, pokud dva různé vstupní řetězce mají stejný hash.

Hlavní část programu bude hledat řetězec, který má částečně stejný nebo úplně stejný hash jako vstupní parametr.

Konkrétně, vstupem programu bude

- `target` - 16 znaků dlouhý hash - hledaný hash
- `overlap` - číslo od 1 do 16 označující, jak velkou kolizní část hashe hledáme

V případě zadání nesprávného vstupu by program měl skončit s chybovou hláškou

```
Invalid parameters
```

Váš algoirmus by měl nalézt řetězec `result` **tisknutelných** ASCII znaků, jehož hash (tzn.

`hash(result)`) obsahuje podřetězec dlouhý alespoň `overlap` znaků, který je také podřetězcem vstupního `target` .

Pokud vstupním parametrem `overlap` bude číslo 16, znamená to, že hledáme řetězec, který má úplně stejný hash jako řetězec, jehož hash je vstupní `target` .

Například na vstup

```
cqcueffuvrmssoxyz
7
```

Hledáme řetězec `result` , jehož hash bude obsahovat podřetězec délky alespoň **7**, který se vyskytuje také jako podřetězec ve vstupním `cqcueffuvrmssoxyz` , což je hash řeteze `Hello World` .

Výsledkem může být například řetězec `kcooitbulc` , jehož hash je `wsyjgbcueffuvwdy` a s puvodním hashem tedy sdílí společný podřetězec `cueffuv` délky 7.

```
cq **cueffuv** rmssoxyz
wsyjgb **cueffuv** wdy
```

Soutěž o největší kolizi

V rámci úlohy je dobré experimentovat, jakou největší kolizi je váš program schopný nalézt.

Vyplňte svůj největší nález na začátek zdrojového souboru (do komentářů), kde popíšete

- jaký byl váš hledaný hash (`target`)
- jakou největší kolizi (`overlap`) se vám povedlo najít
- jakou metodu jste použili pro hledání kolizí

- jak dlouho daný výpočet trval
- a hlavně výsledný řetězec (`result`), jehož hash má takto dlouhý překryv s hledaným hashem (`target`)